CO1 ASSIGNMENT – TASK 1 & TASK 2

Grama Jyothi Rural Electricity Co-operative Billing System

TASK 1 – Understand the Problem and Identify the Data

Explanation: The program reads the previous and current electricity meter readings, calculates units consumed, and handles a 16-bit meter rollover. The meter register ranges from 0 to 65535.

Formula: Units consumed = Current reading – Previous reading. For a single rollover, unsigned 16-bit arithmetic gives the correct result.

Input: Previous reading = 1200; Current reading = 1530.

Output: Electricity units consumed = 330 units.

Rollover test: Previous reading = 65400; Current reading = 120. Output = 256 units.

C Program – task1.c

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (unsigned short)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);

    printf("Consumption: %u units\\n", consumption(1200, 1530));
    printf("Consumption: %u units\\n", consumption(65400, 120));

    return 0;
}
```

Expected Output

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

Explanation of rollover: (65535 – 65400) + 1 + 120 = 256 units. The unsigned 16-bit result wraps around instead of becoming negative.

Data identified: Inputs: previous reading, current reading, household number and category. Outputs: units, energy charge, fixed charge and total bill. Fixed tariff values should be constants. Money should be stored as integer paise to avoid floating-point rounding errors.

Text vs integer representation: An integer reading can be compared and used directly in arithmetic in constant time, O(1). A text reading such as "01530" requires processing its d digits to convert or compare, O(d), and needs space for the characters plus a null terminator.

TASK 2 – Choose the Data Types

Explanation: Choose a data type based on the kind and range of each value. Use unsigned short for the 16-bit meter register, unsigned int for consumption, and an integer type such as long for money stored in paise.

Data item	Suggested C type	Reason
Previous/current reading	unsigned short	16-bit meter values: 0 to 65535
Units consumed	unsigned int	Non-negative consumption
Household number	int	House identification number
Energy charge / total	long	Money stored as integer paise
Tariff rate	int	Rate represented in paise

Input: This program has no keyboard input; it displays the sizes and limits of C data types using sizeof, limits.h and float.h.

C Program – task2.c

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("DATA TYPE SIZES\\n");
    printf("char: %zu bytes\\n", sizeof(char));
    printf("short: %zu bytes\\n", sizeof(short));
    printf("int: %zu bytes\\n", sizeof(int));
    printf("long: %zu bytes\\n", sizeof(long));
    printf("long long: %zu bytes\\n", sizeof(long long));
    printf("float: %zu bytes\\n", sizeof(float));
    printf("double: %zu bytes\\n", sizeof(double));

    printf("\\nDATA TYPE LIMITS\\n");
    printf("USHRT_MAX = %u\\n", (unsigned)USHRT_MAX);
    printf("INT_MAX = %d\\n", INT_MAX);
    printf("LONG_MAX = %ld\\n", LONG_MAX);
    printf("FLT_MAX = %e\\n", FLT_MAX);
    printf("DBL_MAX = %e\\n", DBL_MAX);

    return 0;
}
```

Expected Output (typical 64-bit Linux GCC; your system may show a different long size)

```
DATA TYPE SIZES
char: 1 bytes
short: 2 bytes
int: 4 bytes
long: 8 bytes
long long: 8 bytes
float: 4 bytes
double: 8 bytes

DATA TYPE LIMITS
USHRT_MAX = 65535
INT_MAX = 2147483647
LONG_MAX = 9223372036854775807
FLT_MAX = 3.402823e+38
DBL_MAX = 1.797693e+308
```

Money precision: Do not store rupees as float for exact billing. Store ■2.25 as 225 paise and ■50.00 as 5000 paise. Integer paise avoids binary floating-point rounding in bill totals.

Rollover example: For previous = 65400 and current = 120, the 16-bit unsigned difference is 256. This happens because the 16-bit register wraps from 65535 to 0.

Conclusion: Correct data type selection prevents overflow and precision problems. Use unsigned short for the meter register and integer paise for money.